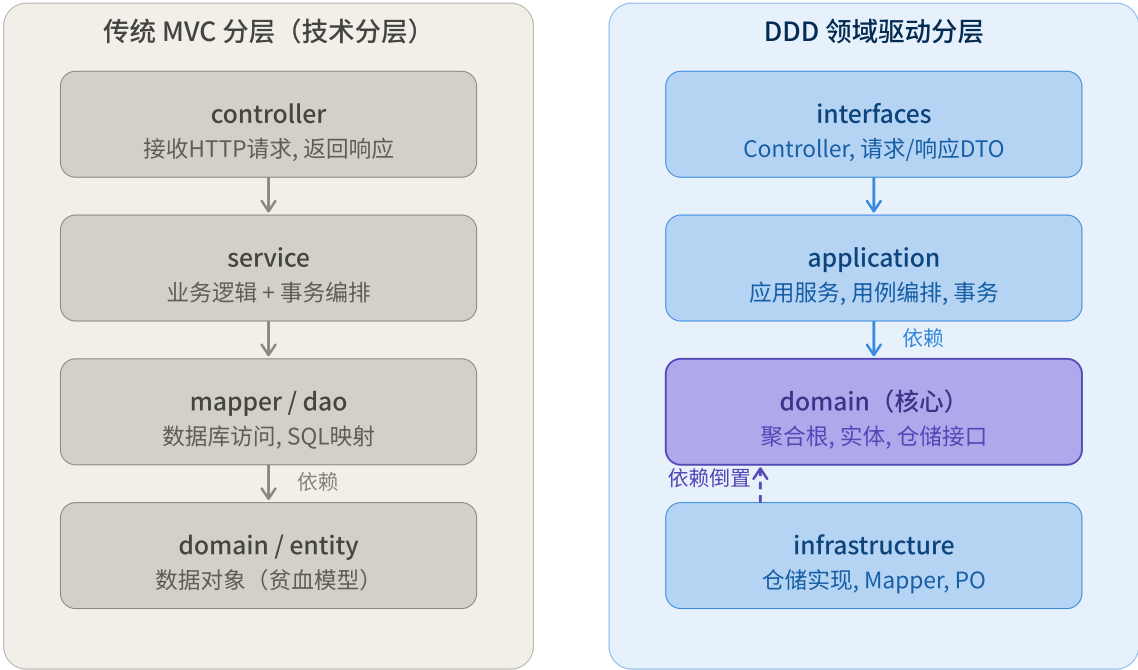


DDD 分层与传统 MVC 分层架构对比

MiniAlalipay 项目架构分析 · 2026年8月3日

两种分层架构对比



核心区别：传统分层依赖向下流动，领域层被动承载数据；DDD 依赖倒置，领域层居中且零框架依赖

上图左半边是本项目当前采用的 DDD 四层架构，右半边是传统 MVC 技术分层。两者看似都有四层，但 **分层依据、依赖方向和职责边界** 完全不同。下面逐一拆解。

一、分层依据不同：按技术职责 vs 按领域语义

传统 MVC 分层——按技术职责切包

每个包对应一种技术关注点：

包	职责
controller	HTTP 入口、参数校验、响应序列化
service	所有业务逻辑（往往堆积成上帝类）
mapper / dao	数据库 CRUD
domain / entity	数据载体（贫血模型，只有 getter/setter）

DDD 四层——按领域语义切层

技术细节被推到边缘：

层	职责
interfaces	进站适配器：REST Controller、请求/响应 DTO、全局异常处理
application	用例编排：应用服务、命令对象、事务边界， 不含业务规则
domain	核心 ：聚合根、实体、值对象、仓储接口、领域错误码， 零框架依赖
infrastructure	出站适配器：仓储实现、MyBatis Mapper、持久化对象 PO

关键差异：传统分层的 domain 只是数据包；DDD 的 domain 是**业务规则的唯一归宿**，包含状态流转、不变量校验、TCC 补偿等全部领域逻辑。

二、依赖方向不同：向下流动 vs 依赖倒置

这是**最本质**的区别。

传统分层——依赖向下流动

controller → service → mapper → domain。上层直接依赖下层的具体实现，service 直接注入 Mapper 接口并调用。如果 mapper 换技术栈（比如 MyBatis 换 JPA），service 必须跟着改。

DDD 分层——依赖倒置 (DIP)

所有箭头指向 domain：

- interfaces 依赖 application
- application 依赖 domain
- infrastructure 依赖 domain（实现 domain 层定义的仓储接口）

项目实例：仓储接口 `CreditAccountRepository` 定义在 `domain/credit/` 下，它的实现 `CreditAccountRepositoryImpl` 在 `infrastructure/credit/` 下，用 `@Repository` 注解注入 MyBatis Mapper。domain 层**不知道** MyBatis 的存在——它只依赖自己定义的接口。这意味着换掉整个持久化方案，domain 层一行代码都不用改。

三、领域模型不同：贫血 vs 充血

传统分层：贫血模型

```
// 传统写法: entity 只有数据
public class CreditAccount {
    private String id;
    private long creditLimit;
    // getter / setter ...
}

// 业务逻辑全在 service
public class CreditAccountService {
    public void freeze(CreditAccount account, long amount) {
        if (account.getCreditLimit() < amount) throw ...;
        account.setCreditLimit(account.getCreditLimit() - amount);
    }
}
```

问题：业务逻辑散落在各个 service 中，同一个实体的规则被多处修改，无法保证不变量。

DDD 分层：充血模型（聚合根封装业务规则）

```
// DDD 写法: 聚合根封装业务规则，以项目 CreditAccount.java 为例
public class CreditAccount {
    private String creditAccountId;
    private long creditLimit;
    private long availableLimit;
    private CreditAccountStatus status;
    private long version;

    // 业务行为直接在领域对象上
    public void freeze(long amount) {
        validateInvariants();           // 不变量校验
        if (availableLimit < amount)
            throw new BusinessException(CreditErrorCode.INSUFFICIENT_CREDIT);
        this.availableLimit -= amount;
        // 状态流转、TCC 冻结逻辑都在这里
    }
}
```

业务规则集中在聚合根内部，application 层只负责“调谁、怎么编排”，不实现规则本身。

四、对象分离不同：一层到底 vs PO 与 Domain 分离

传统分层

`entity` 既是数据库映射对象（`@Table`、`@Column`），又是业务对象，一层到底。好处是简单，坏处是数据库结构变更直接波及业务逻辑。

DDD 分层：引入 PO（Persistent Object）做隔离层

本项目做法如下：

对象	所在层	用途
<code>CreditAccount</code> （聚合根）	domain	纯业务对象，无任何 ORM 注解
<code>CreditAccountPO</code>	infrastructure	数据库映射，对应表结构
<code>CreditAccountMapper</code>	infrastructure	MyBatis 注解 SQL
<code>CreditAccountRepositoryImpl</code>	infrastructure	负责聚合根 ↔ PO 双向转换（ <code>toDomain()</code> / <code>toPO()</code> ）

数据库加字段只改 PO 和 Mapper，聚合根和业务逻辑不受影响。

五、包结构映射对照

传统分层	DDD 分层	变化
controller	interfaces + interfaces/dto	增加请求 DTO、全局异常处理
service	application (编排) + domain (规则)	拆开 : 编排和业务规则分离
mapper / dao	infrastructure/mapper + infrastructure/po	增加 PO 隔离层
entity	domain (聚合根/实体) + infrastructure/po (持久化)	一分为二
无	domain/xxx/Repository (仓储接口)	新增 : 接口在 domain, 实现在 infra

六、项目实际落地情况

从代码看，`account-center` 是唯一完整实现 DDD 全链路的模块，调用链路为：

```
CreditController (interfaces)
  → CreditRepaymentService (application, @Service)
    → CreditAccount / CreditRepayment (domain, 聚合根/实体, 纯POJO)
      → CreditAccountRepository (domain, 接口)
        → CreditAccountRepositoryImpl (infrastructure, @Repository)
          → CreditAccountMapper (infrastructure, @Mapper)
            → CreditAccountPO (infrastructure, PO)
```

`account-center` 的 domain 层按子域拆分为 6 个子包：

子域	包含内容
<code>account</code>	AccountStatus、AccountType 枚举
<code>credit</code>	CreditAccount 聚合根、CreditFreeze、CreditPurchase、CreditReceivable、CreditJobRun 实体、TCC 方法、仓储接口、领域错误码
<code>bill</code>	CreditBill 聚合根、CreditBillItem 实体、账单状态枚举
<code>repayment</code>	CreditRepayment、CreditRepaymentAllocation、CreditRepaymentDraft 实体及仓储接口
<code>ledger</code>	LedgerDirection 枚举（账本方向）
<code>tcc</code>	TccAction、TccBranchStatus 枚举（TCC 事务模型）

其余三个服务（`user-center`、`business-center`、`ai-service`）目前只有 domain 层的枚举骨架，`application / infrastructure / interfaces` 层仅占位 `package-info.java`。

七、总结：为什么选 DDD

维度	传统 MVC	DDD 四层
分层依据	技术职责	领域语义
依赖方向	向下流动	依赖倒置，指向 domain
领域模型	贫血（getter/setter）	充血（封装业务规则）
持久化耦合	entity 直接映射表	PO 隔离，聚合根不感知数据库
可测试性	难（service 依赖 Mapper）	高（domain 纯 POJO，可单测）
换技术栈成本	高（波及业务层）	低（只改 infrastructure）
适合场景	简单 CRUD	复杂业务（金融、交易）

结论：本项目是支付系统，涉及资金、TCC、复式记账、状态流转和幂等——业务复杂度高、不变量多。DDD 的充血模型和依赖倒置能把这些规则集中保护在 domain 层，比传统 MVC 安全得多。代价是**文件更多、转换代码更多**（PO ↔ Domain 转换、仓储接口 + 实现），但对于金融系统来说这个成本是值得的。